The application: shell, screens and flow

The record of the agreed shape of Temco Platform as an application — the regions of the window, the screens, the way one leads to the next, and the two rules the design is built on. It is the design of record: a surface that disagrees with it is either a bug or a proposal to change this file.

Confirmed 2026-10-27. Decisions and their reasons are in *What was decided*; the work order is in *Order of work*.

To **use** the application rather than design it, read getting-started.md: the same flow as numbered steps, with the button for every page and the command id behind it.

The two rules

1. **A VS Code view is native.** The workbench draws a tree view; an extension cannot put Fluent inside one. So the left panel is a Fluent **header + chips + footer** (a webview view) above a **native Pages tree**. The tree keeps reveal, drag, multi-select, keyboard and accessibility for free — the parts where re-implementing costs the most and shows the least.
2. **One project is one window state.** Switching project re-reads and lands on the dashboard. Nothing keeps describing the project that was open before.

The regions

TEXT

```
┌────┬──────────────────┬──────────────────────────────────────────┬──────────────────
│ A  │  LEFT PANEL      │  EDITOR AREA (tabs)                      │  RIGHT PANEL
│ C  │  project card    │  dashboards · forms · the page editor    │  Outline
│ T  │  search          │                                          │  Draft
│ I  │  status chips    │                                          │  Properties
│ V  │  pages tree      │                                          │
│ I  │  build/release   │                                          │
│ T  │                  │                                          │
│ Y  │                  │                                          │
├────┴──────────────────┴──────────────────────────────────────────┴──────────────────
│ OUTPUT · PROBLEMS · TERMINAL        build log · registry diagnostics · the repo's task
├──────────────────────────────────────────────────────────────────────────────────────
│ Contoso T3000 · 2 sources · 148 pages · 3 stale · v1.4 · Build ✓ 10:42 · Pages ✓
└──────────────────────────────────────────────────────────────────────────────────────
```

Activity bar: five entries

	entry	contains	today's views
◉	**Home**	projects, recent, what needs you, Learn	`temcoDashboard` (projects, platform, activity) + `temcoLearn`
▤	**Docs**	the workspace: pages, sources, publish	`temco` (pages, needs-you, sources, publish) + `temcoRepositories.list`
✦	**AI**	chats, models, drafts	`temcoAiHub` + the `temcoAi` secondary sidebar
⚙	**Tools**	SQL Studio, and the modules not written yet	`temcoSql`
⚙	**Settings**	settings, Help	`temcoSettings` + `temcoHelp`

Eight icons became five: three are places to *be* (Home, Docs, AI), two are where you *configure* things. `Repositories` moved into Docs ▸ Sources, `Learn` into Home, `Help` under Settings. No view is deleted.

Implemented (2026-10-27): the five containers exist with those titles, `temcoRepositories.list` is contributed under `temco`, and the `temcoLearn` / `temcoHelp` / `temcoRepos` containers are gone. Container **ids** were kept where they were already used (`temcoDashboard`, `temco`, `temcoAiHub`, `temcoSql`, `temcoSettings`) so saved layout and `onView` activation keep working — only titles and

view membership changed. The guard in `build/check-temco-modules.ts` now refuses a view pointed at a container nobody declares, which is the mistake this consolidation can make silently.

Screens

#	screen	center	left	right
S1	**Home — no project**	hero: New project · Open a source · recent	projects tree	—
S2	**Create project**	3-step form: name/slug/version → where → what is in it	—	—
S3	**Home — project selected**	one callout for the one thing that needs a person, the five-stage pipeline with the figure each stage is answerable for, then *what needs a person* and *what happened lately*	project tree	—
S4	**Add source**	—	inline form in Sources	—
S5	**Docs workspace**	the page editor or a dashboard tab	**Fluent panel**: project card · search · chips · pages tree · build/release footer	Outline · Draft · Properties
S6	**Registry review**	two panes: areas ↔ pages, with issues, and *Fix*	as S5	—
S7	**Page editor**	Temco Editor: one-row bar, paper, status strip	as S5	Outline (default) · Draft · Properties
S8	**New page**	kind · title · area · order · ships → creates file + registry row → **S7**	as S5	—
S9	**Preview / Diff**	beside the editor: the published layout, or the diff against HEAD	as S5	—
S10	**Build**	step list (Resolve → Render → Assets → Sitemap), streaming log, result card	as S5	—
S11	**Release**	version, what ships, destination `releases/<version>/`, a confirmation naming every value	as S5	—
S12	**Deploy to Pages**	repo · address the site will have · what will be committed · **Publish**; never force-push	as S5	—

#	screen	center	left	right
S13	**Export**	format cards: site `.zip` · single-file HTML · Markdown bundle · (`.docx`) · scope	as S5	—
S14	**AI**	conversations, models, the answer with page citations	conversations + runtime	—
S15	**Learn**	the site: documentation · training · tests · products · Q&A · topics, then the reader for one version	the site folder	—
S16	**Settings**	profile · platform (theme, AI runtime, build defaults, Pages branch)	sections	—
S17	**Help**	search, topics, support (logs, diagnostics)	topics	—

Screens S1–S13 are the documentation journey; S14–S17 are the platform around it.

Navigation

from	action	to
anywhere, no project	launch	S1
S1	New project	S2 → S6
S2	Create	S6; the status bar switches to the new project
S5 tree / S3 row	a page	S7 (same tab), split on `Ctrl+click`
S6	a row	S7, or the issue in the page's own Problems
S7	Build	S10 → S11 → S12
S11	Release	S12 (toast), S10 keeps the result
S12	Deploy	stays; status bar `Pages ✓` and the URL opens
S13	Export	stays; the file is revealed in the OS
any	a registry error	S6 with the issue selected
any	the project in view is gone	S1

The publish journey

```
TEXT
```

```
create project —▶ add source —▶ registry —▶ write —▶ validate —▶ BUILD —▶ RELEASE —▶
                                   (S6)      (S7)     (S6)      (S10)     (S11)     (S1
```

- **Build** renders the registry itself: `dist/site/<page>.html` from `marked`, one stylesheet, `sitemap.xml` (only when the GitHub remote gives a real address), `.nojekyll`. A project that already has its own `docs:site` / `docs:build` script keeps using it — the generator is the fallback that makes "a folder of `.md` files" buildable.
- **Release** freezes `releases/<version>/` and records it in the catalog (already built).
- **Deploy** commits the built site to the branch Pages serves and pushes it — a plain, fast-forward push, never a forced one: if the branch holds work this build does not know about, that is reported instead of erased. The commit is made in a throwaway repository whose only parent is the branch's tip, so the project's own working tree is never touched. The one-time Pages switch (`Settings → Pages → Source: branch “gh-pages”, folder “/”`) is attempted through the `gh` CLI when it is installed, and printed as two clicks when it is not. **Why not Actions:** the generator reads the documentation registry, which only exists inside this extension, so a CI job cannot run it. An Actions workflow would need a second copy of the renderer inside the project — a copy that goes stale the first time either side changes. CI publishing becomes possible again the day a project owns its own renderer; the branch push is what makes publishing work until then.
- **Export** is local and dependency-free: the site as a `.zip`, any page as one self-contained HTML file, the pages and their media as a Markdown bundle. The zip writer is in the module (`src/zip.ts`, store + deflate over a CRC32 table) rather than a dependency, because an export a colleague cannot open is not an export. `.docx` needs a writer package (`mammoth` is import-only) and is still owed.

Temco Learn

Temco Learn is its own module (`extensions/temco-learn`, the tenth), because a reader-facing website is not the same job as keeping a page in step with the code. **The website itself is a separate repository** — `temcocontrols/temco-learn`, Vite · React 19 · Fluent UI v9 · Markdown — and this module is a *client* of that checkout: it runs the site's own servers, builds it, shows it, writes new builds into it, and commits the result. Nothing about rendering the site lives here any more, so the site can be worked on, built and published by someone who never opens the editor.

`temcoLearn.siteRepo` names the clone (empty means `~/Temco/temco-learn`). Everything below is a fact about that folder:

- `content/<project>/site.json` — the project's title, section and summary; `content/<project>/<version>/*.md` — the pages a person writes, with front matter for the title;
- `public/imported/<project>/<version>/` — a project's own build HTML, copied in as it came out;

- `public/catalog.json` — the one file both the website and this module read. `npm run catalog` folds the two sources above into it (order, titles, versions, `builtAt`), so the panel cannot list a page the site does not have. A second reader of a second shape is how a panel and a page start disagreeing;
- `package.json` scripts — `client-dev` (Vite, with reload), `build` (catalog + search index + `tsc` + `vite build`), `serve` (the built pages), `catalog`, `search`, `check`. Neither server script names a port: the module passes `--port`, so there is one port for both and one setting that moves it;
- `.github/workflows/pages.yml` — a push at `main` builds the site and hands `dist` to GitHub Pages. The built site is never committed: what is published is the source and the catalog it generates.

The module runs the site, it does not re-implement it. `src/siteRunner.ts` spawns the repository's own `serve` script through `npm`, tracks one server on one port (1025 by default), and treats "starting" as distinct from "running": it polls the address until it answers, so a frame is never pointed at a port with nothing behind it. `Stop` kills the process tree and waits for the port to go quiet, because `npm` hands the server to a child that can outlive the kill. A `dist` that was never built is built before the built pages are first served, rather than served as an empty folder.

`Preview` is a panel framing the running site (`src/preview.ts`, `temcoLearn.site.preview`), with a bar over the frame that is a **caption, not a toolbar**: a tinted strip (`--vscode-sideBar-background`, 34px, one hairline under it) holding the state as a coloured dot and one word, then the address, then the acts — and every one of those is a **link**, not a box, so nothing in the strip competes with the website below it. No address bar, because the address is not a question: one port, set once, and a person who wants the site is not choosing a URL — but the address *is* clickable, because what a person does with a URL is click it, and it goes where the act beside it goes. The three acts read `Reload · Open in Browser · Stop`: the second one is named for where it goes rather than for the noun `Browser`, which said what the thing is instead of what the button does. `Reload` and `Open in Browser` are inert with no address; `Stop` never is. Closing the panel does not stop the server, the way closing a browser tab does not stop a website; `Stop` does. `Add This Build to the Site` copies a build into `public/imported/<project>/<version>/`, writes the `site.json` the catalog needs (the title comes from the build's own `<title>`), runs `npm run catalog`, and shows the new version — only the version is asked, and it is asked once, in the version box, never in a notification. `Publish` commits the clone and pushes it at `main`; it refuses a folder that is not a clone instead of initialising one, because an empty history pushed at a real site is worse than a refusal.

The panel is **three views in the container**, the way Home is three — VS Code draws the header, the separator and the collapse state, so the grouping is the one a person already knows from the Explorer, and each section carries its own title-bar actions. A row is a name and a glyph, and what it holds is `TreeItem.description` — the grey words the Explorer writes beside a folder (`3 versions`), never a badge of our own. Sentences live on the Overview page and in tooltips, never in a row.

view	rows	the grey words
Overview	one row, `Temco Learn`, which opens the page	—
Site	**Preview the Site** (a state dot and one word; its heading opens the Site page; `Stop` / `Start`, `Open` and `Open in Browser :1025` under it), then **Library** (one row per project, at its newest version)	the state: `Running`, `Starting…`, `Stopped`, `Failed`, `Not set up`; projects served
Artifacts	one row per project built on this machine, opening onto the files its build wrote; `Publish` closes the view (`initialSize: 4`)	**nothing** — the name only, and a hover carries the count and the paths; `Publish` puts its count at the row's right end

Three views, one subject each. *Artifacts* is **content**: the files the builds on this machine wrote, plus the single act that is about all of them at once (`Publish`). Nothing here is a task list and nothing here is a state — a view holds things, and the things are files.

Adding a build is not in the panel at all, because it is a *site* operation: it copies a build into the site repository and asks for its version, exactly like cloning, building and publishing write into that repository. So it lives on the **Site page**, as a section of its own — `Builds not on the site`, one row per build with the reason it waits (`not on the site`, `newer than 13.0`) and one `Add…` button — and every command that means "add a build" (`temcoLearn.addBuilds`, from the Overview callout, its workflow card, the Publishing page's step, the `+` on the Site view) opens that page **with the section first and marked**. That is the whole answer to "what is that row for": it is the step that turns a folder on disk into a version on the website, it needs the version typed, and it belongs beside the other steps on the site.

`Build a project` is not a row either: building happens in Temco Docs, so it appears **only when nothing has been built at all** — `Nothing built on this machine yet` / `Build a Project…` — where it is the honest next step. The ellipsis is VS Code's signal that a click leaves this window, and the tooltip says where it goes.

A label is **navigation** or an **instruction**, and the two are not the same sentence: a button on a page that goes to the other module reads `Open Temco Docs` (the way VS Code says `Open Settings`), while the last row of an empty view reads `Build a Project…`, because a view with one line in it has to answer *what do I do*, not *what opens*.

Site answers two questions, in that order. **Preview the Site** is a group whose *heading* is the page and whose *children* are the only acts a panel is good for: `Stop` while it is running and `Start` while it is not (one row, the word swapping with the state), then the two places the site can be read — `Open`, a tab in this window, and `Open in Browser`, which shows the port alone (`:1025`) — and both start the server first if it is down. The two are named for the difference a person feels, not for the API behind them: `Open` stays here, `Open in Browser` leaves for the browser they already have. Everything else a clone

needs — clone, pull, install, build, change the folder or the port, the log — is the Site page's, because a second copy in the panel is how a panel and a page come to disagree about what is running. A clone that is not there swaps those rows for `Clone temco-learn…` and `Use an Existing Clone…`, and the heading says `Not set up` and still opens the page that explains it.

The heading's glyph is the **state**, not the site: a filled circle in `charts.green` while it serves, `sync~spin` while it comes up, `error` in `charts.red` when it failed, `circle-outline` when it is stopped, `cloud-download` in `charts.blue` when there is no clone — the way VS Code's own Ports view draws a running process, so a glance down the panel answers *is it up?* before a word is read.

A row's facts sit at the row's right-hand end, and in a tree that is a decoration badge and nothing else: `description` is drawn inline, immediately after the name, so it can never be a column — but a badge is a `::after` on the label element, which is a flex row, so it is the last flex item and lands at the edge (`views.css` even gives it a 4px right margin). `RowDecorations` answers one badge per row of the module's own scheme: `Running` / `Stopped` / `Failed` / `Not set up` in the state's colour, `:1025` on the browser row, `2 projects` on the library. So the three rows read as a table without a single space character of padding, and the words are in the tooltips as well, for anyone who has turned badges off. The provider answers **synchronously**: a decoration that arrives as a promise leaves the first paint of its row blank, because `DecorationsService` only has the data when it is already there.

Library is one row per **project, at its newest version** (`T3000 · 13.0`), opening that page on the site at `#/<project>/<version>/` and starting the server first if it is stopped. Not one row per version: a product with twenty versions turned the panel into a scroll bar, the newest is the one a person wants nine times in ten, and the site's own list holds the rest. The name is `Library` because the rows are documentation sets, not pages — a *page* lives inside a version, and a *category* is the catalog's `section` field, a different axis. The row writes the project's **name**, without the "docs" a build's own page title usually ends with (`Test1 docs` reads `Test1 · 1.0`), and it carries no description: the version is already in the label, and a grey `2 pages` beside four rows was the noisiest thing in the view. A site with nothing on it does not report emptiness: its one row is `Add a Build to the Site…`, which runs the same picker the view's title bar does. There is deliberately no group of `dist` builds: the build as written is a file on disk, which is the Artifacts page's business, not something to read here.

The Site page is where the clone is managed (`src/sitePage.ts`, `temcoLearn.site.page`), and it is what `Preview the website` opens when there is no clone — a folder dialog was never an explanation. It reads the folder every time it is drawn (`src/siteFacts.ts`: branch, commit, ahead/behind, changed paths, `node_modules`, `dist`) and offers the act for each answer: clone into `~/temco/temco-learn`, use an existing clone, change the folder or the port, pull (`--ff-only`, never a merge), install, build, start, stop, open a terminal, and clear or open the log (`src/siteLog.ts`, the same bounded ring the publish page keeps). The runner grew the two git steps the page needs (`git`, `pull`, `clone`), so a clone and a pull print into the same log a build and a server do.

The page is a control panel, not an article, and it names itself **`Documentation Site`** — `Site` said nothing the view title did not, and `Temco Learn` above it was a breadcrumb of one, which is a label

pretending to be navigation. The title block says **what the site is** (its own repository, the toolchain, one port) and the state (`Running · localhost:1025 · built`, `Stopped`, `Failed`), then **one primary act chosen by that state** — `Clone` → `Install dependencies` → `Build the site` → `Start the site` → `Open`, with `Try again` when a start fails. `Open` goes to a tab in this window; the title carries the `globe` chip, because it is the page of the site and the browser is what it shows.

The site is read in two places, and the page names both the way the panel does: `Open` in the **This machine** row stays here, `Open in Browser` leaves for the browser the reader already has. A button labelled `Open` that opened a browser was the ambiguity worth removing; the tooltip on each says which one it is.

A failure is said **once**, at the top, in the tinted callout — with the reason and the two acts that answer it (`Try again`, `Open the full log`). The title block then says only `Failed`; saying the reason twice made the reader look for the difference between two identical messages.

The clone's own facts are **not in the sentence**. A meta line under the title block carries them, in mono, and pairs them with the two figures that say how far the copy has drifted — `0 commits behind origin`, `5 paths changed` — so they scan as numbers with left edges in a column rather than hiding at the end of a paragraph. The repository, the commit and the changed paths are **links** (a command, never an `href`): the first two open GitHub at that commit, the last opens the Publishing page, which is where those paths are listed. A clone with nothing changed links nothing — a link to an empty list is a dead end:

```text
temcocontrols/temco-learn · main · 577ca57 · committed 3 days ago
0 commits behind origin        5 paths changed
```

Below that, the page is two parts, and both section titles wear the brand's vertical rule (the shape the Overview page uses):

part	what it answers	drawn as
This machine	what is here, and is it running	the server as one line — state in mono, `Repository` | `Terminal`, then `Start` / `Stop`, `Open` and `Open in Browser` — then four **small cards in one row**: Folder · Port · Installed · Built (two across under 760px). A card is a label, a value and the act that changes it, **with no border**: its condition is a 6px dot in `charts.green` when the step is done, `charts.yellow` when it is not, and a hollow one when there is no condition to report
Activity	what it said	a tabbed area, `Output` and `Publish`, plain text on a tint off the card surface: no border, no header bar, no line numbers. The tabs are radios and labels, so they switch with no script and still answer the keyboard

`Log` was the wrong word, and so was `Console` — a log is a file, and this is what the module *did*: a clone, a pull, a build, a server, and the last publish. It used to be two places as well: pressing `Build` opened the preview panel to report on itself, so a build either pushed the site out of the way or the message was lost. Every message now goes to the page that owns the button (`say` / `complain` in `src/extension.ts`), which is why the area has two tabs: `Output` is this window's command lines, `Publish` is the publish page's log, which a person used to have to leave the page to read.

Every line is cleaned before it is drawn (`cleanLine`): ANSI colour codes stripped (an escape sequence arrives as `ΓÇö` in a console that is not a terminal), control characters dropped, the module's own `[site]` / `[git]` prefixes and the `$` prompt removed, and the glyphs a build or a dev server uses to decorate its output (`➜`, `✓`, box drawing) taken out — the words they decorate are kept. The raw lines stay in the output channel, where a terminal is the right place for them.

One port, one way of serving. `serve` listens on 1025 (`temcoLearn.sitePort`, above 1024 so it binds without root on macOS and Linux) and shows `dist/` as it was last built. There is no second mode: `client-dev` — Vite with hot reload — exists for editing the site's own React and Markdown, which is the repository's business and not something this page decides, so a terminal still runs it (`npm run client-dev -- --port 1025`). A page that serves what was built has one way to serve it, and a switch that changes nothing for the reader is a question nobody asked. A change to the port takes effect the next time the server starts, and the log says so.

Publishing is one page and one press (`src/publishPage.ts`, `temcoLearn.publishing`). The row reads `Publish the Site` — the count is already at its right-hand end as a badge, so the label says what is published and the number stays a number. The row opens the page rather than acting on the click, and the page has four sections, each with the 2 px brand bar:

- **The repository** — one card, not a band of four cells. A head line of **links** (the remote opens the repo on GitHub, the commit opens that commit, the branch sits plain between them), then `12 paths in this commit` and the first **five** of them, each a link to the file on GitHub. The rest fold into a native `<details>` — `Show 7 more paths` — so a publish that touches forty files reads as six lines and expands in place, with no script and no round-trip. The foot says whether anything is unpushed and offers `Open the clone` · `Terminal`.
- **What publishing does** — the four step cards, each ending in the act that performs it.
- **Where it goes** — one card for the one subject: `ADDRESS` as the page's biggest **link** (it opens the live site), then `Served from` (main, built by GitHub Actions, handed to Pages), `Last publish` and `Waiting` as label/value pairs.
- **The last attempt** — the lines git printed, unchanged.

The button that does it (`temcoLearn.publishNow`) writes its result where the decision was made, so there is no preview strip to read it in and no second page to be redirected to.

Artifacts has a page (`src/downloadPage.ts`, `temcoLearn.downloads`), opened by the Artifacts view's empty row and by its title bar. Three sections, each with the 2 px brand bar the other pages carry: **What**

a build writes (five cards — the website, the help file, the pdf, the archive, the Word document), **Where it lands** (one card, because there is one working area: the folder written out as the card's own **link**, and two lines spelling out `<project>/dist` and the `HelpDocs` folder), and **The files on this machine** (a grid of **file cards** per project, packed **from the left** at the width the content asks for — `minmax(215px, 240px)` with `justify-content: start`, because letting them stretch put one file in a card 900px wide — each with the glyph for its kind, the name, `offline help · 1.1 MB`, and the two acts a file on disk has: `Show in the folder` for every card, and `Save a copy` where a copy can be taken). `Look` used to sit there and drew the viewer into the inventory; looking at a `.pdf` or the built site is the panel's job and the OS's, so the card finds the file instead and gets out of the way.

The page lists only what a build produces **as a whole**. A loose `.html` in `dist` is a page someone was iterating on, not an output: listing it beside `T3000.pdf` made the page read like a folder dump, so it is filtered out — and a project whose only result is such a fragment is not a project here at all, which is why the count in the title block counts the cards rather than the files on disk. The panel's Artifacts view draws the same line: it lists what can be taken away, never a fragment. **Publish** closes that view as `Publish the Site` — the count sits at the row's right-hand end as a badge, and the tooltip carries the last publish. Publishing is about every build at once, which is why it is the one act that sits with them.

Nothing is invented: a project with no `.chm` has no `.chm` row, and the panel says `Nothing built on this machine yet` only when nothing exists at all. Publishing state is read from git (`src/state.ts`) in the clone, so two machines never disagree about what is out.

Answers appear in the **preview's own strip**, under its bar, and never in a notification: adding a build, building, publishing, and every failure say what happened there, with at most one thing to do next. Only the *inputs* stay native — the version box, and the build picker when a row did not name its build, and the folder dialog for a clone that has not been chosen yet.

Settings → General is a bundled Fluent page (`webview/settings/` , React 19 + Fluent UI v9), not markup generated in the extension host: the tokens are Fluent's, the brand is Azure's blue, and every control is a real Fluent control. It is drawn in the house page language rather than a settings template — the brand wash with an eyebrow, a title and a note; one tinted callout; a section heading with the 2px Azure rule before it; an entry row per setting with its control at the right-hand end; a facts band; and links at the foot of the last block. `main.tsx` imports `temco-ui` 's `media/page.css` , so the wash, the rule, the row and the buttons are the same rules Temco Learn's Overview page is drawn by and the two pages cannot drift.

The host's half is narrow: `src/settingsPages.ts` builds the bundle when a source tree has none (a compile wipes `out/`), hands the webview `settings.js` and `settings.css` under a content-security policy that allows nothing else, and answers three messages — `ready` with one state, `setting` with a key to write, `command` to run. A refresh is a new state and never a new document, so a write does not throw away the reader's scroll. `npm run build-webview` builds it; `build-webviews` and `vscode:prepublish` run that too.

Design system

- **Fluent v9** (`@fluentui/react-components` , `@fluentui/react-icons`) with the brand ramp seeded from Azure blue `#0078D4` through `createLightTheme` / `createDarkTheme` , following VS Code's light, dark and high-contrast through `data-vscode-theme-kind` . The package named *Azure theme* (`@fluentui/azure-themes`) is v8/Fabric and does not apply.
- **Tokens**: spacing 4 / 8 / 12 / 16 / 24 · radius 4 / 6 / 10 · type 20 / 16 / 14 / 13 / 11 · row 22 (compact 20) · status colours taken from the VS Code chart colours, so a chip, a dot and the status bar cannot disagree.
- **Row anatomy**, one component reused everywhere: `[icon] title · meta · status dot · hover actions` .
- **Page anatomy**: the title block (one `h1` , one line of why, the ways in), then sections whose titles carry a 2px brand bar to their left, then at most **one** tinted callout, which is the thing a person would do next. Figures live beside the thing they are about — a stage of a pipeline, a row in a list — never in a strip of tiles whose only label is the number itself.
- **A list folds, it does not truncate**: the first five rows are shown and the rest are behind a native `<details><summary>…</summary>` , so the count in the heading stays true.

What was decided

question	decision
activity bar	five: Home · Docs · AI · Tools · Settings
left panel	Fluent header, search, chips and footer in a webview view; the Pages tree stays native
build	our own generator, so a project with only `.md` files builds; a project's own `docs:site` still wins
deploy	build in the editor, push to `gh-pages` with a plain (never forced) push; the one-time Pages switch via `gh` when available
export	site `.zip` , single-file HTML, Markdown bundle; `.docx` optional
order of work	generator → deploy → export → panel and shell

Order of work

1. **Generator** ✅ — `src/siteRender.ts` (pure: slugs, page shell, index, sitemap, one stylesheet), `src/site.ts` (host: read, render, write, copy media), `scripts/check-site.mts` (10 cases); `Build the Site` falls back to it.
2. **Deploy** ✅ — `src/pages.ts` (the remote decides the sitemap's address) and `src/git.ts` shared with the diff command. The push of a site lives in `temco-learn` (`src/publish.ts` : `ensureRepo`

refuses a folder that is not a clone, then a plain, never-forced `git push origin HEAD:main`; `scripts/check-publish.mts`, 5 cases against real throwaway repositories); `temcoDocs.publishSite` stays behind as a redirect. Still owed: the status-bar state and the confirmation screen S12.

3. **Export** ✅ — `src/zip.ts` (a real archive writer: store, deflate, CRC32), `src/export.ts` (site zip, single-file page, Markdown bundle; `temcoDocs.export` + three commands), `scripts/check-export.mts` (5 cases that read the archive back). Still owed: `.docx`. Then the panel.

4. **Panel and shell** — *(in progress)* the activity bar is five entries ✅; the **Project** strip above the native Pages tree ✅ (`src/panelView.ts`, `check-panel.mts`); the **status bar** ✅ (`src/statusLine.ts`, `check-status.mts`); **S3** (the project dashboard) ✅ inside the Project Detail page: pages · drafts · stale · issues, a *Needs you* group, and *Last out of the building*; **S13** ✅ as the **Exports** page (`temcoDocs.export` opens it; the three shapes are still their own commands, plus `.docx` from `src/docxWrite.ts` — Word's format is a zip of XML, so it needs no dependency); **S12** ✅ folded into the Releases page as *Where It Publishes*; **S1** ✅ the Projects page opens itself when the window is on no project and has no tab; **S6** ✅ as *The Registry* page — every entry with the reason it might not appear in the site, and one action per reason; **S10** ✅ the generator reports its steps (Resolve → Render → Assets → Sitemap) to the notification and the output channel as they happen; **S11** ✅ publishing a release confirms first, naming version, destination and file count.

   Phase 4 is done, and so is every gap the screen audit found. What remains for the shell is the tabbed *Ship* surface (Build · Release · Deploy · Export as one page) — deliberately not built: after the cross-links and the Exports page, the four surfaces point at each other, so it is polish rather than a gap. The status line deliberately carries the window's facts (project · pages · issues) and not the version or the last build: those need the catalog, and a catalog read on every file save is too high a price for a status bar. The strip in the side bar shows them instead, from a render that already had a reason to read the catalog.

Overview is an application page (`src/overview.ts`, view type `temcoLearn.overview`), drawn with `temco-ui` the way every other page of ours is, and it is never published. It opens with the sentence that says what the module is for — *Temco Learn publishes the documentation website* — continued by the counts it reads from the snapshot (*3 projects are published today, 7 versions and 42 pages between them*) so no project name is ever written into the prose. Then, in this order: one callout for what needs doing (or that nothing does, or that nothing has been built yet); **Publishing workflow**, four cards — *Build · Add · Publish · Download* — one line each, ending in the link that performs it (`Open Temco Docs`, `Choose a build · 1 waiting`, `Publish · 3 changes`, `Show in the folder`); and **Where it stands**, three lines, one per page that acts on the facts they name. The section headings carry the 2 px brand bar the other pages use.

And it stops there — with one line back. The page used to go on with three parts — *Published projects*, *Build outputs* and *Publication* — that restated what the pages which act on those facts already say: the Site page holds the clone, the versions it serves and where it is published; the Artifacts page holds what

a build wrote; the Publishing page holds the address, the branch and what a push would carry. A summary page that copies the page it summarises drifts from it, so the **lists** went. What came back is **Where it stands**, and it is deliberately one line per page rather than a section per subject:

line	the fact on it	it opens
`Documentation Site`	`2 projects · 3 versions · 4 pages`	the Site page
`Artifacts`	`3 files · 5.3 MB`	the Artifacts page
`Publishing`	`https://temcocontrols.github.io/docs/ · 3 changes waiting`	the Publishing page

A **count** is the one thing worth repeating: it is computed from the same snapshot on every draw, so it cannot drift, and it answers "is there anything in there?" without a list that has to be kept in step. Each line ends in the standard `Open ›`, so the detail is one click away and lives where it is edited.

Add a build is a section of the **Site page**, not a panel view: it belongs to the page that clones, builds, runs and publishes the site, because it writes into the same repository those do. `Builds not on the site` lists every build on disk whose project is absent from the catalog or whose `dist/site` is newer than the version the catalog lists, each with the reason (`not on the site`, `newer than 13.0`) and one `Add…` button. Adding asks for the version, copies into `public/imported/`, runs `npm run catalog`, and publishes nothing. Every command that means "add a build" opens that page with the section first and marked. Nothing about it is automatic: a test build stays out of the public site until somebody adds it. A link on this page is text with a glyph (`temcoLearn.*` command), never an `href` into the website — that boundary was crossed once already. The module therefore depends on `temcocontrols.temco-ui`, exactly as `temco-docs` does.

A build is not one thing. `temco-docs` writes the site (`dist/docs` or `dist/site`), an offline help file (`.chm`, inside the repository or in a `HelpDocs` folder beside it), often a `.pdf`, and the exports it keeps in `dist/` — a zip of the site, a Word document, single pages. `src/artifacts.ts` reads those three places per project and reports what is there: nothing is invented, so a project with no `.chm` simply has no `.chm` row, and a file in a shared `HelpDocs` folder belongs to the project it is named after. Each artifact is either something to look at (`site`, `chm`, `pdf`, a single page) or something to take away (`zip`, `docx`, and the files that are both), which is the divide the panel draws.

The panel (`src/tree.ts`) is three views in the order the questions are asked — **Overview** (one row: the page above), **Site** (`Preview the Site` with a state dot and the port, and **Library** with one row per project at its newest version), and **Artifacts** (a project, and the files its build wrote — names only, with the size and the path on hover; a row saves a copy and offers *Show in the Folder* inline; `Publish the Site` closes the view, its count at the row's right end). What a row holds is its `description`: the grey words beside a name, as the Explorer writes them — and where a row deliberately says nothing, the hover carries it.